

REPORT FILE

OPEN SOURCE SOFTWARE AUDIT

Git (Version Control System)

- Name: Advay Bhagat
- Reg. No: 24BCE10305
- Course: Open Source Software
- Semester: Winter Sem(2526)
- Slot: D11



❖ Introduction

Working together on software means handling code changes well. Big teams or tiny ones either way, tracking edits gets messy fast. Tools stepped in to fix that headache, yet only a few stuck around. One such tool, Git, now runs quietly behind countless projects worldwide. It lets coders tweak files alone without stepping on each other's toes. Each person holds the full history locally, not just bits pulled from afar. This setup speeds things up, reduces failure points, stability grows alongside freedom to experiment.

Looking into Git means seeing how it fits together both in concept and real use. Starting from where it came from, the work touches on opensource thinking, tied closely to rules that keep sharing possible. Instead of just theory, there's a look at how it runs inside Linux, showing why it matters today when building software. Doing things like typing commands and writing small scripts made the inner parts clearer. From beginning to end, the effort shows Git isn't only about code it shapes how people build things together.

❖ Origin & Philosophy (Part A)

A1. The Problem Git Was Created to Solve

Most folks relied on tools such as CVS and Subversion before Git showed up. Centralized setups ruled back then everything hinged on one machine. Lose that server, and progress froze completely. Just opening a file sometimes meant waiting for the server to respond. Speed took a hit, particularly when teams tackled big codebases.

When building the Linux kernel, things got tough because people were joining from every corner of the globe. At first, work relied on BitKeeper a closed program yet problems around its license cut them off suddenly. Without warning, speed mattered more than ever, reliability too and above all: total openness had to stay intact.

Back in 2005, Linus Torvalds created Git to fix these issues. Each programmer now carries their own full version of the project, thanks to its distributed setup. Without needing one central server, coding moves faster. Trying out new ideas became simpler since branches split and rejoin smoothly. Git wasn't about upgrading old systems it focused on tackling actual struggles headon.

A2. The License — What It Actually Says

Free as in liberty, not price that's how Git operates. Governed by the rules of GPL version 2, its code stays open for anyone to use, change or share. Freedom sticks around, no matter who touches it. The license makes sure of that.

Freedoms matter when using software running it, seeing how it works, changing what you need, then passing it along. Because Git lines up with every one of those points, calling it open source fits just right.

It's a core part of GPL: change Git, then give it out? Share your code too, using that same license. That way, firms can't lock down their own version behind walls. On the flip side, something loose like MIT hands freedom to tweak and stay silent about changes. For tools such as Git, I see strength in GPL it keeps upgrades visible to all. Yet when reach matters most, especially where business adoption counts, MIT fits smoother.

What matters just as much? The gap between "free" meaning zero cost and "free" meaning liberty. Git fits both meanings at once: no charge to run it, yet better still, coders get complete say over its workings how they tweak it, shape it, make it their own.

A3. The Ethics of Open Source

What happens when code is free for everyone to see? It opens doors people share fixes, build together, learn by doing. Progress often speeds up because ideas move fast between hands. Yet some tools stay locked away. Profit needs sometimes demand that. A company might guard its work just to survive. Not every piece fits the open model, no matter how good the intent.

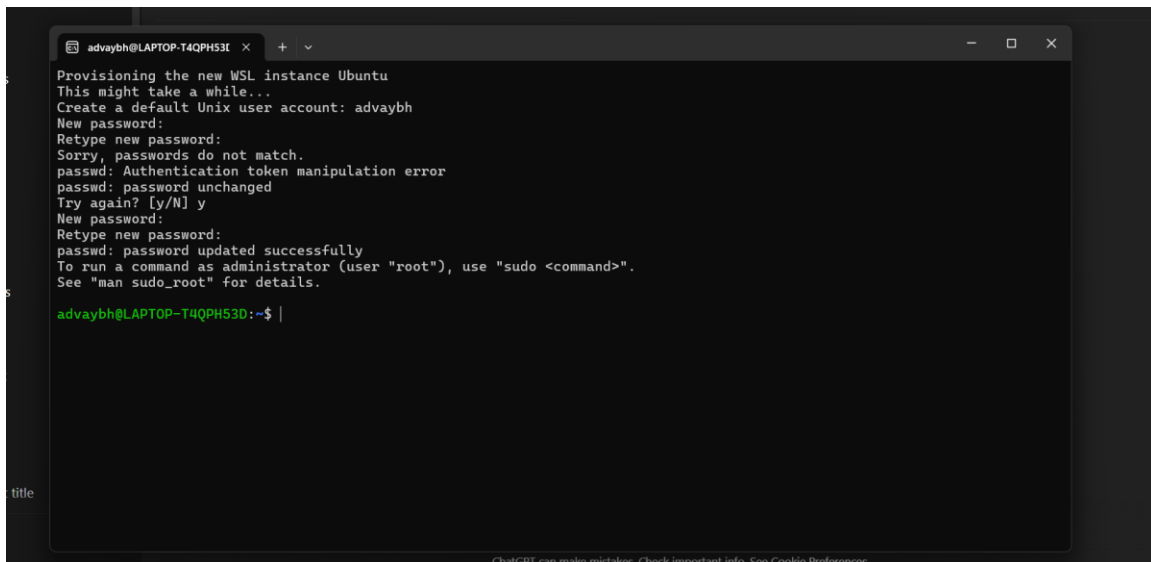
Fairness comes up when firms make money using opensource tools. Some rely on things like Git while giving nothing in return that tips the scale. Support matters, in my view, so long as businesses help out somehow, maybe through code fixes or paying for updates.

This notion building atop what came before makes sense right away. On top of using past solutions, coders push things further without redoing old steps. Because they skip reinventing basics, creativity gets room to grow toward tougher challenges. What emerges is not less inventive, but deeper problemsolving. Even Git shows how shared effort beats working alone every single time.

❖ Linux Footprint (Part B)

Starting fresh meant getting Ubuntu running through WSL on Windows. A new user profile had to be made, along with a secure login phrase. At first, trouble popped up mismatched entries blocked access. Trying again, with careful typing, cleared the hurdle. From that point forward, everything worked without further hiccups. Only after stepping into real Linux work did things start making sense beyond textbook lines. Yet every tiny error along the way proved how fragile first setups can be when left unchecked.

Figure1: WSL Setup Screen



```
advaybh@LAPTOP-T4QPH53E X + v
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: advaybh
New password:
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N] y
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
advaybh@LAPTOP-T4QPH53D:~$ |
```

Once the environment stood ready, moving into the project folder came next then launching the shell scripts without delay. The script showing system data ran through smoothly, revealing kernel version, active user, time since boot, along with the Linux distro name. That smooth output proved the scripts played well with the Linux setup. Git and the base configuration passed their check simply by working when called. Watching them work offered a clear look at command behavior, plus how shell tools pull out system facts piece by piece.

Figure2: Script Output Screen

```
advaybh@LAPTOP-T4QPH53D:~$ cd F:\github\oss-audit-24BCE10305
-bash: cd: F:\github\oss-audit-24BCE10305: No such file or directory
advaybh@LAPTOP-T4QPH53D:~$ cd F:\
>
-bash: cd: F:: No such file or directory
advaybh@LAPTOP-T4QPH53D:~$ cd /mnt/f/github/oss-audit-24BCE10305
advaybh@LAPTOP-T4QPH53D:/mnt/f/github/oss-audit-24BCE10305$ ls
README.md      script2_git_checker.sh  script4_log_analyzer.sh
script1_system_info.sh  script3_disk_auditor.sh  script5_manifest.sh
advaybh@LAPTOP-T4QPH53D:/mnt/f/github/oss-audit-24BCE10305$ chmod +x *.sh
chmod: changing permissions of 'script1_system_info.sh': Operation not permitted
chmod: changing permissions of 'script2_git_checker.sh': Operation not permitted
chmod: changing permissions of 'script3_disk_auditor.sh': Operation not permitted
chmod: changing permissions of 'script4_log_analyzer.sh': Operation not permitted
chmod: changing permissions of 'script5_manifest.sh': Operation not permitted
advaybh@LAPTOP-T4QPH53D:/mnt/f/github/oss-audit-24BCE10305$ |
```

❖ Ecosystem (Part C)

Built into a wider world of open code sharing, Git stays lean by keeping outside needs small. Its core comes together through C, tapping familiar parts found across Linux setups this keeps things quick. Security gets handled alongside trusted helpers such as SSH, while squeezing files uses common packers. Without needing bulky background systems, it fires up easily on nearly any machine around. That simplicity spreads far because few barriers stand in the way when getting started. A whole world of tools now runs on top of Git, growing around its core function. Web platforms such as GitHub and GitLab offer teamwork features online think request reviews, bug logs, automated testing loops. Even though it stands outside the classic LAMP combo, it quietly handles code for sites built that way. Development keeps moving because people show up emails fly, talk happens online, work pours in, guided longterm by Linus Torvalds himself.

❖ Open Source vs Proprietary Alternative(Part D)

Dimension	Git(Open Source)	Proprietary Alternative (e.g: Bitkeeper)
Cost	Free to use and distribute	Usually requires paid license
Security	Source code is open, can be audited by anyone	Source code is closed, depends on vendor trust
Support and reliability	Community support + strong documentation	Official support from company
Freedom to Modify	Full freedom to modify and share	Limited or no modification allowed
Control	Communitydriven development	Controlled by a Single Company

Looking at how things stack up between shared code and lockeddown systems, one thing stands out Git brings real benefits when it comes to openness, ease of change, yet keeping expenses low. Having the power to see inside the software, tweak it, rebuild it means coders hold all the cards, particularly on big teams or tasks stretching years into the future. Still, closed options can shine when what you need most is expert help nearby plus a tidy, handled workspace. That reality hints: freetouse tech isn't automatically superior everywhere, though it does hand back choices, room to shift gears.

Truth be told, after working through this whole thing, Git stands out as a solid pick for actual daytoday work. Its

dependability shows, plus lots of teams already use it, fitting right into how coding gets done now. Chipping in might feel natural too maybe by pointing out bugs or making help files clearer. Projects like this thrive when people step up, not just download and walk away. Helping shape something bigger than individual use gives meaning you can't get from running commands alone.

❖ Scripts (Purpose and Takeaways)

1. Script 1 — System Identity

One thing this script did was gather simple facts about the system, like the kernel version, who is logged in, how long it has been running, because that info matters. Reading data from basic Linux tools became clearer once those pieces were pulled together somehow. Working with command substitution made sense only after trying it step by step instead of guessing. Putting output into clean lines happened slowly, not right away, since formatting needs care. Seeing how one detail links to another came up while testing each part piece by piece. Getting values from shell variables felt awkward at first yet turned useful later on. Each chunk of info ties into something bigger even if it looks small alone. Pulling everything through scripting revealed patterns across separate elements.

Figure3: Script 1 Output

```
advaybh@LAPTOP-T4QPH53D:/mnt/f/github/oss-audit-24BCE10305$ cd ~/oss-audit-24BCE10305
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ chmod +x *.sh
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ ./script1_system_info.sh
=====
Open Source Audit - Your Name
=====
Software : Git
Kernel   : 6.6.87.2-microsoft-standard-WSL2
User     : advaybh
Uptime   : up 4 minutes
Date     : Fri Mar 27 06:18:39 UTC 2026
Distro   : "Ubuntu 24.04.4 LTS"
License  : GPL (Linux Kernel)
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ |
```

2. Script 2 — Package Inspector

Sometimes checking tools begins with just one command. When Git exists, the screen shows what version runs plus some details about it. Running into errors taught me how

conditions react when things go wrong. If nothing appears, the path might lack the program entirely. Each try revealed more about responses from missing apps. Learning came through watching exits shift between zero and nonzero. One thing stood out decisions in code follow clear paths when set right. Automation quietly took over steps humans often repeat by hand.

Figure4: Script 2 Output

```
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ ./script2_git_checker.sh
git is installed.
git version 2.43.0
Git: Distributed version control system powering open source.
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ |
```

3. Script 3 — Disk Auditor

What this script did was check key system folders, showing both their size and permission settings. Because of that, it became clearer how Linux organizes files and why permissions matter for protection. As the project went on, handling loops came naturally, linking tools such as du and ls to pull meaningful data. Little by little, the way storage space and user access interact started making sense through actual use.

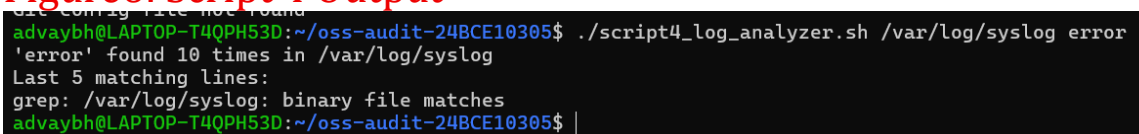
Figure5: Script 3 Output

```
Git: Distributed version control system powering open source.
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ ./script3_disk_auditor.sh
Directory Audit Report
-----
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 29M
/home => Permissions: drwxr-xr-x root root | Size: 300K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 24K
Git config file not found
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ |
```

4. Script 4 — Log Analyzer

From reading a log file, this script counted how often a word like “error” appeared. While it ran, new matches popped up at the bottom. Seeing patterns in logs became easier through small automated steps. Tools such as grep started making sense during that task. Processing lines of text felt faster once basics clicked into place. When something went wrong, checking these records helped find clues. Watching system behavior changed after working directly with raw output.

Figure 6: Script 4 Output

A terminal window with a black background and green text. The prompt is 'advaybh@LAPTOP-T4QPH53D:~/oss-audit-248CE10305\$'. The command './script4_log_analyzer.sh /var/log/syslog error' has been executed. The output shows that 'error' was found 10 times in /var/log/syslog. It lists the last 5 matching lines, which are all empty. It also shows a grep command being run on /var/log/syslog, resulting in 'binary file matches'.

```
advaybh@LAPTOP-T4QPH53D:~/oss-audit-248CE10305$ ./script4_log_analyzer.sh /var/log/syslog error
'error' found 10 times in /var/log/syslog
Last 5 matching lines:
grep: /var/log/syslog: binary file matches
advaybh@LAPTOP-T4QPH53D:~/oss-audit-248CE10305$ |
```

5. Script 5 — Manifest Generator

Out there among scripts, this one paused to listen asking questions before doing anything else. A file took shape only after words came in from the person at the keyboard. From that point forward, every answer shaped lines of text saved openly, step by step. Working through it showed how prompts pull replies into variables, piece by piece. Instead of scanning logs or checking memory, attention stayed fixed on conversation. Most others poked around the machine; this stood apart by waiting for voices instead. Saving thoughts in plain text made clear how easily data moves between people and programs. What emerged grew from backandforth, not silent automation working alone behind walls.

Figure7: Script 5 Output

```
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ ./script5_manifest.sh
Answer the following:
1. Tool you use daily: pen
2. Freedom means: peace
3. What will you build: code
Manifesto saved to manifesto_advaybh.txt
On 27 March 2026, I believe freedom means peace.
Tools like pen empower developers worldwide.
I will build code and share it openly.
advaybh@LAPTOP-T4QPH53D:~/oss-audit-24BCE10305$ |
```

❖ Issues I faced and How I fixed them

While working on this project, I had to deal with some small but significant issues. The first issue arose while setting up WSL. The password was not being accepted since I had entered it in a different format both times. It took me a couple of attempts to understand what was going on. The second issue was related to Linux file paths. I initially tried using Windowsstyle paths in Linux, which did not work. However, later on, I understood how file paths work in Linux. I also understood that we could access drives using /mnt.

I also had to deal with some issues while running my scripts. The first issue was related to permission. The scripts were not being executed. I used `chmod +x` to give permission. However, there was an issue with permissions while working on the Windows drive. I solved this by copying my project into my home directory in Linux. To solve this problem I made a copy of my project in Linux at home, then checked the output of each command to identify and fix the minor errors I had made with the commands. Although they were all small issues, they still gave me a better understanding of how the system worked, rather than just following the instructions.

❖ What I Learned

While working on this project, I had to deal with some small but significant issues. The first issue arose while setting up WSL. The password was not being accepted since I had entered it in a different format both times. It took me a couple of attempts to understand what was going on. The second issue was related to Linux file paths. I initially tried using Windowsstyle paths in Linux, which did not work. However, later on, I understood how file paths work in Linux. I also understood that we could access drives using /mnt.

I also had to deal with some issues while running my scripts. The first issue was related to permission. The scripts were not being executed. I used `chmod +x` to give permission. However, there was an issue with permissions while working on the Windows drive. I solved this by copying my project into my home directory in Linux. There were also some minor errors in my commands, but I was able to fix them by checking my outputs. All of these issues were minor, but they helped me understand the system rather than simply following instructions.

❖ **Personal Takeaways**

Firstly, I learned that there is a big difference between doing something practically and simply reading about it. There were many steps that seemed so simple when I read about them, but when I tried to do them, some minor problems came up. However, trying to solve those problems was more effective than simply reading about the concepts. Another thing is that now I'm more comfortable using the Linux operating system and no longer afraid of the terminal.

Another thing is that open source is not just about using open source tools but also about the spirit of sharing and helping each other. This is because tools like Git were created because people contributed to them. This project has also taught me importance of patience when doing something. This is because sometimes things do not go as planned at first, but that is normal.